Coinsult

Request your audit at coinsult.net

Advanced Manual
Smart Contract Audit

March 24, 2026

x.com/coinsultaudits

t.me/coinsult_tg

Audit requested by

Zylo Vortex | Airdrop

0x0000000000000000000000000000000000000000

Global Overview

Manual Code Review

In this audit report we will highlight the following issues:

Vulnerability Level	Total	Pending	Acknowledged	Resolved
Informational	2	0	1	1
Low-Risk	3	0	0	3
Medium-Risk	5	2	0	3
Critical-Risk	5	0	0	5

Risk Classification

Coinsult uses certain vulnerability levels, these indicate how bad a certain issue is. The higher the risk, the more strictly it is recommended to correct the error before using the contract.

Vulnerability Level	Description
Informational	Does not compromise the functionality of the contract in any way.
Low-Risk	Won't cause any problems, but can be adjusted for improvement.
Medium-Risk	Will likely cause problems and it is recommended to adjust.
Critical-Risk	Will definitely cause problems, this needs to be adjusted.

Audit Summary

Project	Zylo Vortex | Airdrop
Website	https://powerunit.live/
Blockchain	BSC (BEP-20)
Source Code	https://bscscan.com/address/0x0000000000000000000000000000000
Contract Address	0x0000000000000000000000000000000000000

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Audit Scope

Coinsult was commissioned to perform an audit based on the provided code. Note that we only audited the code available to us on this URL at the time of the audit. If the URL is not from any block explorer (main net), it may be subject to change. Always check the contract address on this audit report and compare it to the token you are doing research for.

Audit Method

Coinsult's manual smart contract audit is an extensive methodical examination and analysis of the smart contract's code that is used to interact with the blockchain. This process is conducted to discover errors, issues and security vulnerabilities in the code in order to suggest improvements and ways to fix them.

Automated Vulnerability Check

Coinsult uses software that checks for common vulnerability issues within smart contracts. We use automated tools that scan the contract for security vulnerabilities such as integer-overflow, integer-underflow, out-of-gas-situations, unchecked transfers, etc.

Manual Code Review

Coinsult's manual code review involves a human looking at source code, line by line, to find vulnerabilities. Manual code review helps to clarify the context of coding decisions. Automated tools are faster but they cannot take the developer's intentions and general business logic into consideration.

Used tools

Slither: Solidity static analysis frameworkRemix: IDE Developer ToolCWE: Common Weakness EnumerationSWC: Smart Contract Weakness Classification and Test CasesDEX: Testnet Blockchains

Table of Contents

Global Overview 2

Audit Summary 3

Audit Scope 4

Detailed Findings

Owner can change start time (breaks rewards) 6

Unit index not validated (OOB revert/DoS) 7

Index not validated in claim (OOB revert) 8

Block details can underflow and revert 9

Claim accounting mismatch enables overclaim 10

tx.origin restriction breaks wallets/relayers 11

Pseudo-randomness is manipulable 12

External dependency can change reward logic 13

Unbounded loops may run out of gas 14

Event emits wrong values (misleading logs) 15

No validation of ZyloPowerUp address 16

Time constants look like test values 17

Potential division by zero if blockSecond set 18

No actual token transfers in airdrop/claim 19

Redundant/unused variables and confusing logic 20

Disclaimer 21

Owner can change start time (breaks rewards)

Critical-Risk *- Will definitely cause problems, this needs to be adjusted.*

Error Code	Description
AUD-001: Resolved	Owner can arbitrarily change inceptStartTime, which changes month/day/block calculations and therefore users' accrued rewards and caps. This enables retroactive reward reduction/increase and can invalidate expected payouts, creating a strong centralization and trust risk.

Code Snippet

```
function updateTime(uint256 _inceptStartTime) external onlyOwner{
    inceptStartTime = _inceptStartTime;
}
```

Recommendation

Remove updateTime or restrict it (one-time set, only before launch). If needed, add timelock + event + bounds checks (e.g., cannot move backwards/forwards beyond small drift). Document governance controls clearly.

Note

No updateTime()/setter exists; inceptStartTime is set once in constructor from ZyloPowerUp and never changed.

Unit index not validated (OOB revert/DoS)

Critical-Risk *- Will definitely cause problems, this needs to be adjusted.*

Error Code	Description
AUD-002: Resolved	_unit is used to index fixed-size arrays powerUpUnitX/getUnitMax without bounds checks. Passing _unit >= 6 will revert, allowing user-triggered denial of service for their own calls and breaking integrations/frontends that don't sanitize inputs.

Code Snippet

```
uint256 unitMax = getUnitMax[_unit];
uint256 _unitAssetX = powerUpUnitX[_unit];
```

Recommendation

Add require(_unit < 6, "Invalid unit") in all external/public entry points that accept _unit (powerUpAirDrop, claimSelfAirDrop, view helpers if desired).

Note

External entry points validate unit: require(_unit < MAX_UNIT) in powerUpAirDrop(), claimSelfAirDrop(), getCurrentAirDropAmount().

Index not validated in claim (OOB revert)

Critical-Risk *- Will definitely cause problems, this needs to be adjusted.*

Error Code	Description
AUD-003: Resolved	claimSelfAirDrop reads userPowerUpDetails[_user][_unit][_index] without checking _index < length. Invalid indices revert, which can break UIs/relayers and makes the function fragile to user input mistakes.

Code Snippet

```
powerUpDetails storage c = userPowerUpDetails[_user][_unit][_index];
```

Recommendation

Add require(_index < userPowerUpDetails[_user][_unit].length, "Invalid index"). Consider helper functions to fetch valid indices.

Note

claimSelfAirDrop() now checks index bounds: require(_index < userPowerUpDetails[_user][_unit].length).

Block details can underflow and revert

Critical-Risk *- Will definitely cause problems, this needs to be adjusted.*

Error Code	Description
AUD-004: Resolved	getBlockDetails computes elapsed = block.timestamp - inceptStartTime. If owner sets inceptStartTime in the future (or constructor value is wrong), subtraction underflows and reverts, breaking all functions that depend on it (powerUp, claim, views).

Code Snippet

```
uint256 elapsed = block.timestamp - inceptStartTime;
```

Recommendation

Require inceptStartTime <= block.timestamp when setting/updating. In getBlockDetails, handle future start time gracefully (return zeros) or revert with a clear error.

Note

Constructor enforces startTime <= block.timestamp and getBlockDetails() returns zeros when block.timestamp <= inceptStartTime, avoiding underflow.

Claim accounting mismatch enables overclaim

Critical-Risk *- Will definitely cause problems, this needs to be adjusted.*

Error Code	Description
AUD-015: Resolved	In internalCall, checkClaimedUnitAmount is increased by claimAmount, but c.selfClaimed is increased by selfpowerup (pre-cap). If selfpowerup > remainingCap, user can repeatedly call claim and each time c.selfClaimed jumps by the larger selfpowerup while cap-tracking uses claimAmount, causing inconsistent state and potential repeated claims/DoS of accounting.

Code Snippet

```
uint256 claimAmount = selfpowerup;
if (claimAmount > remainingCap) claimAmount = remainingCap;
checkClaimedUnitAmount[_user][_unit][_index] += claimAmount;
...
c.selfClaimed += selfpowerup;
```

Recommendation

Update selfClaimed by claimAmount (not selfpowerup). Also compute burnToken from claimAmount, and ensure all claim-related state uses the same capped amount.

Note

internalCall() now increments c.selfClaimed by claimAmount (capped), aligning with checkClaimedUnitAmount and preventing mismatch/overclaim accounting.

tx.origin restriction breaks wallets/relayers

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-005: Resolved	Using msg.sender == tx.origin blocks smart contract wallets (Gnosis Safe), account abstraction, and relayers. It reduces usability and can lock out legitimate users. It is not a reliable security control and can harm composability.

Code Snippet

```
require(msg.sender == tx.origin,"External Access Not Allowed");
```

Recommendation

Remove tx.origin checks. If you need bot protection, use signatures (EIP-712), allowlists, rate limits, or per-user nonces. Support smart wallets for better security and UX.

Note

No tx.origin checks remain; access is based on msg.sender only.

Pseudo-randomness is manipulable

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-006: Resolved	getRandomAssets uses block.timestamp and block.prevrandao, which can be influenced by block producers and is predictable to some extent. If assetsNo affects value/rarity, users/miners can game outcomes.

Code Snippet

```
keccak256(abi.encodePacked(block.timestamp, block.prevrandao, msg.sender)) % 15 + 1;
```

Recommendation

If randomness matters economically, use a verifiable randomness source (e.g., Chainlink VRF) or commit-reveal. If it's cosmetic, document that it's not secure randomness.

Note

Randomness removed; assetsNo is assigned via deterministic getNextAsset() sequence.

External dependency can change reward logic

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-007: Pending	Rewards depend on external contract ZyloPowerUp (airDropAmount, dailyPowerUpAmount, dayReqOnStart, inceptStartTime). If that contract is upgradeable or controlled, it can change user entitlements and calculations, creating centralization and integrity risk.

Code Snippet

```
uint256 _amount1 = ZyloPowerUp.airDropAmount(_user, _unit);
uint256 dailypowerup = ZyloPowerUp.dailyPowerUpAmount(c.powerUpMonth);
```

Recommendation

Document trust assumptions. Consider immutability (immutable address), governance timelocks, or snapshot critical parameters locally at deposit time (e.g., store daily rate per month when powerUp occurs).

Note

Core reward math still depends on external ZyloPowerUp.dailyPowerUpAmount() and airDropAmount() snapshots; if ZyloPowerUp changes behavior/params, user entitlements change.

Unbounded loops may run out of gas

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-008: Pending	getTotalSelfPowerUp/currentSelfPowerUp iterate over all userPowerUpDetails entries. As entries grow, these calls can become too expensive and revert when called on-chain by other contracts, and can be slow even off-chain.

Code Snippet

```
for (uint256 i = 0; i < length; i++) {
    _powerup += getSelfPowerUpbyUnitIndex(_user, _unit, i);
}
```

Recommendation

Keep these as off-chain view helpers only, or add pagination (start/end indices). Avoid designing any state-changing logic that relies on iterating over unbounded arrays.

Note

currentSelfPowerUp() still iterates over the full userPowerUpDetails[_user][_unit] array (unbounded). Pagination exists only for getTotalSelfPowerUpPaginated().

Event emits wrong values (misleading logs)

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-009: Resolved	ClaimSelfAirDrop event is emitted with parameters (_user, _unit, remainingCap) but the event expects (user, amount, unit). This produces incorrect logs, confusing users and indexers and making audits/accounting unreliable.

Code Snippet

```
emit ClaimSelfAirDrop(_user, _unit, remainingCap);
```

Recommendation

Emit in correct order and values, e.g., emit ClaimSelfAirDrop(_user, claimAmount, _unit). Add tests to validate event fields.

Note

Claim event emission now matches values: emit ClaimSelfAirDrop(_user, _unit, claimAmount).

No validation of ZyloPowerUp address

Low-Risk *- Won't cause any problems, but can be adjusted for improvement.*

Error Code	Description
AUD-010: Resolved	Constructor does not validate _zyloPowerUp. If set to zero or non-contract, calls like inceptStartTime() will revert, bricking deployment or causing runtime failures.

Code Snippet

```
constructor(address _zyloPowerUp) {
    ZyloPowerUp = IZyloPowerUp(_zyloPowerUp);
}
```

Recommendation

Add require(_zyloPowerUp != address(0)) and optionally check code length (require(_zyloPowerUp.code.length > 0)).

Note

Constructor validates ZyloPowerUp address is non-zero and has code (code.length > 0).

Time constants look like test values

Low-Risk *- Won't cause any problems, but can be adjusted for improvement.*

Error Code	Description
AUD-011: Resolved	secondsInOneDay/week/month are set to 120/840/3600 seconds, not real-world durations. If deployed to production, reward timing will be drastically different than expected, impacting user payouts and trust.

Code Snippet

```
uint256 public secondsInOneDay = 120;
uint256 public secondsInOneWeek = 840;
uint256 public secondsInOneMonth = 3600;
```

Recommendation

Use real durations (1 days, 1 weeks, 30 days) for production or make them immutable/configurable with strict governance + timelock. Clearly separate test vs mainnet configs.

Note

Time constants use real durations: 1 days, 1 weeks, 30 days (immutables).

Potential division by zero if blockSecond set

Low-Risk *- Won't cause any problems, but can be adjusted for improvement.*

Error Code	Description
AUD-012: Resolved	getBlockDetails divides by blockSecond. It is currently 1 and not changeable, but if future code adds a setter or upgrade, setting it to 0 would brick calculations and revert many functions.

Code Snippet

```
blockNumber = elapsed / blockSecond;
uint256 blocksInMonth = secondsInOneMonth / blockSecond;
```

Recommendation

If blockSecond ever becomes configurable, enforce require(blockSecond > 0). Consider making it immutable/constant if it should never change.

Note

blockSecond is constant = 1, not configurable, so division-by-zero cannot occur in this version.

No actual token transfers in airdrop/claim

Informational *- Does not compromise the functionality of the contract in any way.*

Error Code	Description
AUD-013: Acknowledged	Functions named powerUpAirDrop/claimSelfAirDrop only update internal accounting and emit events; they do not transfer ERC20 tokens. Users may assume they receive tokens when they do not, leading to confusion and support issues.

Code Snippet

```
airDropAmountPowerUP[_user][_unit] += _amount;
...
// no ERC20 transfer
```

Recommendation

Clarify in naming/docs (e.g., “accounting”/“points”). If tokens should be distributed, integrate safe ERC20 transfers and ensure funding, access control, and reentrancy protections around transfers.

Note

Still no ERC20 transfers; functions only update internal accounting and emit events.

Redundant/unused variables and confusing logic

Informational *- Does not compromise the functionality of the contract in any way.*

Error Code	Description
AUD-014: Resolved	Variables like totalpowerup are set then reduced but not used meaningfully; tokenAmount is read but only used for cap; totalpowerup -= selfpowerup has no effect. This increases audit risk and chance of future bugs.

Code Snippet

```
uint256 totalpowerup = selfpowerup;
...
totalpowerup -= selfpowerup;
```

Recommendation

Simplify logic and remove unused variables. Add comments/tests for intended math (cap, burnToken, activeToken updates) to reduce maintenance risk.

Note

Previously confusing claim math was refactored; claim uses claimAmount consistently and updates selfClaimed/burn based on capped amount.

Disclaimer

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Coinsult is not responsible if a project turns out to be a scam, rug-pull or honeypot. We only provide a detailed analysis for your own research.

Coinsult is not responsible for any financial losses. Nothing in this contract audit is financial advice, please do your own research.

The information provided in this audit is for informational purposes only and should not be considered investment advice. Coinsult does not endorse, recommend, support or suggest to invest in any project.

Coinsult can not be held responsible for when a project turns out to be a rug-pull, honeypot or scam.